

TP — Migration DevOps SalLeEnFrance

De Docker Compose à un cluster Kubernetes 2 nœuds — mTLS, CI/CD et Terraform

Cours d'architecture logicielle & déploiement — M2 Ingénierie Web

2025–2026

Table des matières

Avant-propos	2
Contexte métier	2
Compétences visées	2
Modalités	3
Conventions du document	3
Architecture cible	3
L'écosystème Kubernetes — panorama des stacks	4
Distributions Kubernetes (le « kernel »)	5
Outils d'observation et de pilotage (la « surface utilisateur »)	6
Maillage de service (mTLS « à grande échelle »)	7
Repérage rapide	7
Étape 0 — Outillage	7
Étape 1 — Comprendre l'existant Docker Compose	8
Étape 2 — Mapper Docker Compose → Kubernetes	9
Étape 3 — Provisionner le cluster avec Terraform	9
Étape 4 — Connecter Freelens et explorer	10
Étape 5 — Containeriser les services	11
Étape 6 — Premiers manifests : Namespace, Config, Secrets	12
Étape 7 — Stateful : PostgreSQL	12
Étape 8 — Cache : Redis	13
Étape 9 — Microservices	13
Intermède — Comprendre les types de Service	14
ClusterIP (par défaut)	14
NodePort	15
LoadBalancer	16
ExternalName (mention)	16
Tableau de synthèse	17
NodePort vs LoadBalancer — la confusion classique	17
Et l'Ingress dans tout ça ?	17
Étape 10 — Ingress	18
Étape 11 — mTLS inter-services avec cert-manager	19
11.1 Mise en place de la PKI	19
11.2 Certificats par service	19
11.3 Activation côté serveur Next.js	20
11.4 Activation côté client (appels inter-services)	20
11.5 Démonstration	20
Étape 12 — CI/CD GitHub Actions	21

Étape 13 — Observabilité, résilience, rolling updates	22
13.1 Casser pour comprendre	22
13.2 Mise à jour sans interruption	22
13.3 Logs	22
Bonus	22
Annexe A — Cheatsheet kubectl	23
Contexte et namespace	23
Inspection	23
Logs	23
Exécution / debug	23
Application et rollback	23
Scale et autoscale	24
Stockage	24
Secrets et ConfigMaps	24
Diagnostic accéléré	24
cert-manager	24
Ingress	24
Nettoyage rapide	24
Annexe B — Critères de réussite	24
Annexe C — Pièges classiques (à lire avant de commencer)	25

Avant-propos

Contexte métier

SalleEnFrance est une entreprise fictive. Elle commercialise un outil SaaS de réservation de salles de réunion pour des sociétés ayant plusieurs sites en France (Paris, Lyon, Marseille, Bordeaux, Lille...).

L'application existe et fonctionne. Elle est aujourd'hui déployée sur un seul serveur via une stack docker-compose. L'entreprise grandit : la direction technique veut bénéficier de la scalabilité, de la résilience, et d'un déploiement reproductible offerts par Kubernetes.

Vous êtes l'équipe DevOps. Votre mission : industrialiser la plateforme — provisionner un cluster Kubernetes, migrer la stack, sécuriser les flux internes en mTLS, mettre en place une chaîne CI/CD, et vous donner les moyens d'observer et de déboguer ce que vous opérez.

Compétences visées

À la fin du TP, vous saurez :

1. Lire et déconstruire une architecture docker-compose, et identifier ses primitives.
2. Établir une table de correspondance entre les concepts Docker Compose et les ressources Kubernetes.
3. Provisionner un cluster Kubernetes par le code (Terraform), avec deux nœuds.
4. Containeriser des services applicatifs en suivant les bonnes pratiques (multi-stage, non-root, image minimale, healthcheck).
5. Écrire des manifests Kubernetes : Namespace, ConfigMap, Secret, Deployment, StatefulSet, Service, Ingress, PersistentVolumeClaim, NetworkPolicy.
6. Mettre en place une PKI interne et activer le mTLS entre microservices avec cert-manager.

7. Industrialiser la livraison via une pipeline GitHub Actions (lint, scan, build, push, deploy).
8. Observer un cluster en exploitation : `kubectl`, probes, logs, événements, et l'IDE Freelens.

Modalités

- Modalité de rendu : binôme. Un dépôt Git par binôme, accompagné d'un rapport `RAPPORT.md`.
- Le présent document ne contient pas les solutions — il pose les objectifs, les contraintes et les critères de validation. Les YAML du dépôt comportent des points # TODO à compléter ; charge à vous de les compléter intelligemment.

Conventions du document

Objectif. ce qui doit être atteint à la fin de l'étape.

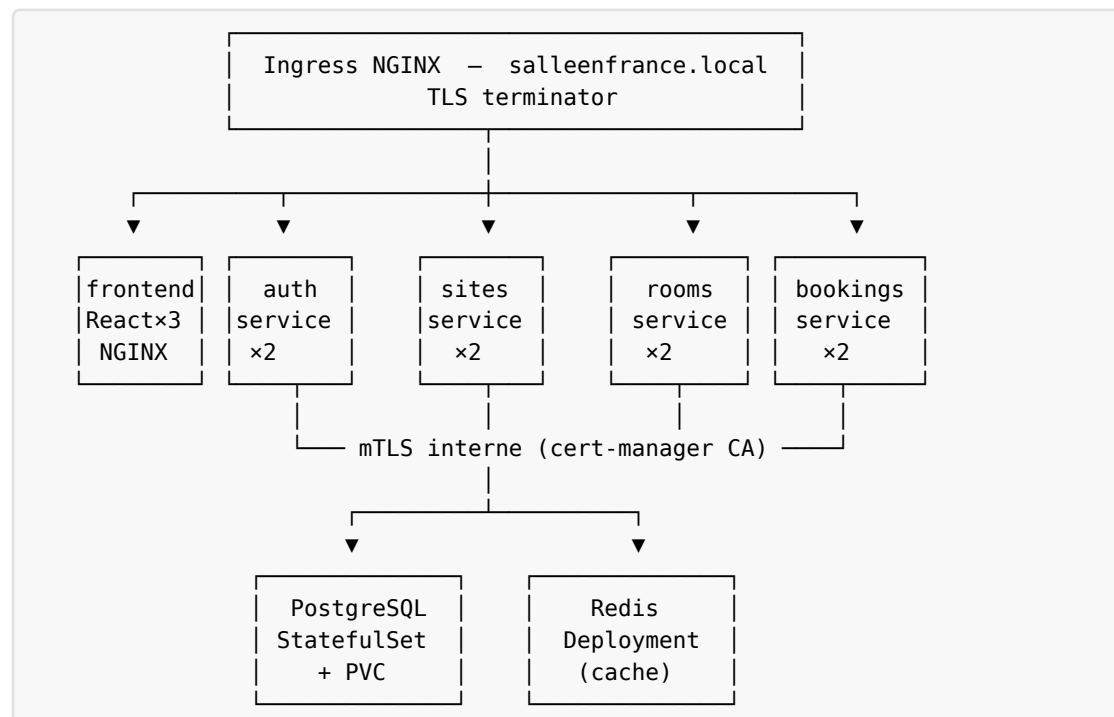
Contraintes. règles non-négociables (sécurité, naming, ressources).

Livraison. le fichier ou la sortie attendue dans votre dépôt.

Validation. ce que vous devez voir / vérifier dans Freelens pour considérer l'étape réussie.

Pièges. erreurs classiques qui vous feront perdre du temps.

Architecture cible



Composant	Rôle	Type K8s	Répliques
frontend	UI React (Vite, statique servi par NGINX)	Deployment	3
auth-service	Authentification, JWT, profils	Deployment	2
sites-service	CRUD sites France	Deployment	2
rooms-service	CRUD salles d'un site	Deployment	2
bookings-service	Réservation et calendriers	Deployment	2
postgres	Persistance	StatefulSet + PVC	1
redis	Cache (créneaux disponibles)	Deployment	1
ingress-nginx	Reverse-proxy TLS	Deployment (chart Helm)	—
cert-manager	PKI interne, gestion des certificats	Deployments	—

L'écosystème Kubernetes — panorama des stacks

Avant de plonger dans l'outillage opératoire, prenez la mesure du paysage. Kubernetes n'est pas *un* logiciel, c'est une API et un ensemble de distributions qui l'implémentent, chacune avec ses choix de packaging, de cible matérielle et de finitions opératoire. Ce chapitre est de la culture générale — aucun livrable demandé, mais bon à connaître.

Distributions Kubernetes (le « kernel »)

Distribution	Cible	Mémoire à retenir
Kubernetes (vanilla) — kubeadm	Référence amont, prod on-prem	La spec officielle. Lourd, on l'installe rarement à la main en prod.
kind	Dev local, CI	Kubernetes IN Docker — chaque nœud est un conteneur. C'est ce que vous utilisez dans ce TP.
k3d	Dev local	Wrapper autour de k3s dans Docker. Plus léger que kind, communauté active.
k3s	Edge, IoT, on-prem light	Kubernetes ré-empaqueté en un seul binaire par Rancher. SQLite par défaut au lieu d'etcd. Très utilisé sur Raspberry, en magasin, sur des passerelles.
k0s	Edge, on-prem	Concurrent de k3s par Mirantis ; binaire unique, sans dépendance hôte (pas même iptables).
MicroK8s	Dev local Linux, IoT	Distribution Canonical en <i>snap</i> . Activation des modules par <code>microk8s enable ...</code> .
OpenShift (OKD)	Entreprise on-prem / cloud	K8s + Red Hat (Operators, Routes au lieu d'Ingress, SCCs, builder). Stack opinionnée et complète.
Rancher	Multi-cluster on-prem	Pas une distrib K8s à proprement parler : un <i>control-plane</i> de <i>control-planes</i> qui gère plusieurs clusters (downstream k3s/RKE/EKS).
Tanzu (VMware)	Entreprise vSphere	Intégration profonde avec vSphere ; pertinent en milieu bancaire / industriel.
EKS (AWS)	Cloud managé	API K8s upstream + control-plane managé AWS. Vous gérez les workers (ou Fargate).
GKE (Google)	Cloud managé	Le plus mature des trois ; mode <i>Autopilot</i> en serverless.
AKS (Azure)	Cloud managé	Bonne intégration AAD / ACR ; mode <i>Automatic</i> récent.
Scaleway Kapsule / DigitalOcean DOKS / OVH MKS	Cloud managé européen	Alternatives moins coûteuses, souvent suffisantes.

Outils d'observation et de pilotage (la « surface utilisateur »)

Outil	Rôle	Quand l'utiliser
kubectl	CLI officielle	Toujours. Tout le reste dérive ou s'appuie sur elle.
Freelens (<i>imposé dans ce TP</i>)	IDE/desktop multi-cluster	Vue d'ensemble, debug visuel, navigation par objet. Fork open-source de Lens depuis le passage en source-disponible.
Lens Desktop	IDE/desktop	Ancêtre de Freelens, désormais payant pour usage commercial. Cité pour info.
K9s	TUI plein-écran	Quand vous vivez dans le terminal. Plus rapide qu'un IDE, raccourcis vim-friendly. À recommander aux SRE.
kubectx / kubens	Bascule contexte / namespace	Si vous opérez plusieurs clusters. Indispensable.
kustomize	Templating sans templating	Inclus dans kubectl <code>apply -k</code> . Préférez-le pour les overlays multi-environnements.
Helm	Gestionnaire de paquets K8s	Pour packager vos apps en chart, et pour installer le tiers (ingress-nginx, cert-manager, prometheus...).
stern	Tail de logs multi-pods	Plus pratique que <code>kubectl logs -f</code> sur un workload répliqué.
kubectl-debug / nicolaka/netshoot	Debug réseau	Lancer un pod éphémère équipé de tcpdump, dig, curl.
cmctl	CLI cert-manager	Statut, renouvellement, troubleshooting des certificats.
Scaffold / Tilt / DevSpace	Boucle de dev rapide	<i>Hot reload</i> dans le cluster. À évoquer pour le bonus dev experience.
Argo CD / Flux	GitOps	Le référentiel Git devient la source de vérité du cluster. Concurrent de l'approche <code>kubectl apply</code> en CI.
Argo Rollouts / Flagger	Déploiement progressif	Canary / blue-green pilotés par métriques.
kube-prometheus-stack	Observabilité	Prometheus + Grafana + Alertmanager + dashboards.
Loki + Promtail	Logs centralisés	Compagnon de Grafana pour les logs.
Falco	Runtime security	Détection d'anomalies (syscalls, files).
Trivy Operator / kube-bench / kube-hunter	Audit de sécurité	Scan continu, conformité CIS.
Velero	Backup / restore	Snapshots namespace ou cluster entier.

Maillage de service (mTLS « à grande échelle »)

Mesh	Mémoire à retenir
Linkerd	Léger, simple, mTLS automatique entre pods. À privilégier en première installation.
Istio (mode <i>ambient</i> récent)	Le plus puissant et le plus complexe. <i>Ambient</i> a beaucoup réduit la complexité historique.
Cilium Service Mesh	Bâti sur eBPF, pas de sidecars. Performant.
Consul Connect (HashiCorp)	Si vous êtes déjà dans l'écosystème Consul.

Dans ce TP vous mettez en place le mTLS manuellement avec cert-manager — c'est le bon réflexe pédagogique pour comprendre. En production, on bascule en général sur un mesh.

Repérage rapide

Mise en situation : « *Pour un cluster d'edge déployé sur 50 caisses enregistreuses dans des supermarchés, vous proposez quoi ?* ». Réponse attendue : k3s ou k0s + GitOps via Flux. Pas EKS (pas de cloud sur place). Pas vanilla (trop lourd pour du matériel modeste). Pas OpenShift (licence + ressources).

Mise en situation : « *Vous voulez offrir le même service à 30 dev qui codent sur leur Mac.* ». Réponse attendue : kind ou k3d, scriptés via Make. Surtout pas un cluster cloud par développeur.

Étape 0 — Outillage

Objectif. disposer d'un poste de travail capable de provisionner et opérer un cluster Kubernetes local.

Contraintes.

- vous installez les outils vous-mêmes, sans ticket support ;
- vous documentez dans `RAPPORT.md` la version exacte de chaque outil ;
- aucune dépendance globale `npm` n'est tolérée — utilisez `npx` ou les `package.json`.

Liste des outils requis :

Outil	Pourquoi	Source
Docker Desktop / OrbStack	runtime conteneur	docker.com / orbstack.dev
kubectl ≥ 1.30	CLI K8s	kubernetes.io
kind ≥ 0.22	cluster K8s local	kind.sigs.k8s.io
terraform ≥ 1.7	infra as code	hashicorp.com
helm ≥ 3.14	charts (ingress-nginx, cert-manager)	helm.sh
freelens ≥ 1.0	IDE de visualisation K8s	freelens.app
cosign	signature d'images (CI)	sigstore.dev
trivy	scan de vulnérabilités	aquasec.com
stern	tail de logs multi-pods	stern.dev

Livrable. section *Outillage* du RAPPORT.md avec la sortie de `kubectl version --client`, `kind --version`, `terraform -version`, `helm version`, `freelens --version`.

Pièges. ne pas mélanger Docker Desktop et Rancher Desktop ; vérifier que `docker context` pointe sur un seul daemon.

Étape 1 — Comprendre l'existant Docker Compose

Objectif. démarrer la stack `docker-compose` fournie, l'utiliser comme un utilisateur final, et en faire la cartographie technique.

Contraintes.

- vous n'avez pas le droit de modifier le code applicatif ; vous opérez sur l'infra ;
- tous les services doivent être joignables et la création d'une réservation doit aboutir.

Livrable.

1. capture d'écran de l'application fonctionnelle ;
2. schéma (draw.io / Excalidraw) listant pour chaque conteneur :
 - son image, sa version, sa taille ;
 - les ports publiés et internes ;
 - les volumes utilisés ;
 - les variables d'environnement (en marquant les sensibles *sensible*) ;
 - les flux réseau entre services (qui appelle qui).

Validation. non applicable à cette étape (Docker Compose, pas K8s).

Pièges. `depends_on` simple n'attend que le démarrage du conteneur, pas sa disponibilité applicative ; pour ce dernier, il faut `depends_on.<service>.condition: service_healthy`

(avec healthcheck). Identifier les *racas* possibles au démarrage est explicitement attendu dans votre rapport.

Étape 2 — Mapper Docker Compose → Kubernetes

Objectif. avant d'écrire la moindre ligne de YAML K8s, formaliser la correspondance conceptuelle entre les primitives Docker Compose et les ressources Kubernetes. Cette étape est purement intellectuelle ; aucun code n'est écrit.

Livrable. un tableau dans votre `RAPPORT.md` à compléter, structuré ainsi (la première ligne est un exemple, à enrichir) :

Concept Docker Compose	Équivalent Kubernetes	Différence sémantique notable
<code>services:</code> (un service compose)	Deployment (ou StatefulSet) + Service	en K8s, l'unité de scalabilité est le Pod, pas le conteneur ; le Service est une abstraction réseau séparée
<code>image:</code>	...	...
<code>ports:</code>	...	...
<code>volumes:</code> (bind mount)	...	...
<code>volumes:</code> (volume nommé)	...	...
<code>environment:</code>	...	...
<code>env_file:</code>	...	...
<code>depends_on:</code>	...	...
<code>healthcheck:</code>	...	...
<code>restart:</code>	...	...
<code>networks:</code>	...	...
<code>secrets:</code>	...	...
<code>deploy.replicas</code>	...	...
<code>deploy.resources.limits</code>	...	...
<code>command:</code> / <code>entrypoint:</code>	...	...

Validation. ce tableau doit figurer complet dans le `RAPPORT.md` et la cohérence des choix doit être justifiable.

Pièges. confondre Service (réseau) et « service » au sens compose ; oublier qu'un Service K8s n'embarque pas la charge — c'est juste une IP virtuelle stable.

Étape 3 — Provisionner le cluster avec Terraform

Objectif. à partir du dossier `terraform/` du dépôt, écrire la configuration nécessaire pour qu'un simple `terraform apply` produise un cluster Kubernetes 2 nœuds (1 control-plane

+ 1 worker) prêt à recevoir des charges, avec l'ingress controller et cert-manager déjà installés.

Contraintes.

- le cluster cible est kind (Kubernetes IN Docker) — provider `tehcyrx/kind` ;
- un nœud du cluster (typiquement le control-plane, étiqueté `ingress-ready=true`) publie les ports 80 et 443 sur l'hôte via `extra_port_mappings` ;
- le state Terraform reste local (`terraform.tfstate`) — n'utilisez pas de backend distant pour le TP ;
- `ingress-nginx` et `cert-manager` sont installés via le provider `helm` dans le même `terraform apply` (deux modules ou deux `helm_release` dans le root module) ;
- aucune commande `kubectl create` ou `helm install` n'est tolérée pour bootstrapper l'infrastructure — tout doit être déclaratif.

Livrable.

- `terraform/main.tf` complété ;
- `terraform/variables.tf` avec au minimum `cluster_name`, `kubernetes_version`, `ingress_chart_version`, `cert_manager_chart_version` ;
- `terraform/outputs.tf` exposant le path du kubeconfig généré ;
- sortie de `terraform plan` et `terraform apply` collée dans le rapport.

Validation.

- dans Freelens, le cluster apparaît avec 2 nœuds Ready ;
- le namespace `ingress-nginx` contient un pod controller Running ;
- le namespace `cert-manager` contient 3 pods Running (controller, webhook, cainjector).

Pièges. le provider `kind` reconstruit le cluster si vous changez `node_image` ou `extra_port_mappings` — relire la doc du provider avant de modifier ces champs sous peine de tout perdre.

Étape 4 — Connecter Freelens et explorer

Objectif. utiliser Freelens comme outil de référence pour observer le cluster tout au long du TP. Chaque étape suivante comportera une checklist Freelens.

Contraintes.

- Freelens lit le kubeconfig produit par Terraform ;
- vous devez garder Freelens ouvert pendant toutes les manipulations ;
- vous n'utilisez pas le terminal interne de Freelens pour appliquer du YAML — l'application se fait via `kubectl` ou via la CI ; Freelens reste un outil de lecture et de debug.

Livrable. capture Freelens montrant la liste des nœuds avec leurs labels (role), leur OS image et leur version kubelet.

Auto-formation. prenez 15 minutes pour parcourir, dans Freelens :

- vue *Nodes* (CPU/RAM allouée vs utilisée) ;
- vue *Workloads / Pods* (filtrage par namespace) ;
- vue *Network / Services* et *Network / Ingresses* ;
- vue *Storage / PersistentVolumeClaims* ;
- vue *Config / ConfigMaps* et *Config / Secrets*.

Pièges. Freelens cache l'état pendant 15s par défaut — si une ressource a l'air figée, forcez le refresh (⌘R) avant de soupçonner un bug.

Étape 5 — Containeriser les services

Objectif. produire des images Docker propres pour chacun des 5 services applicatifs.

Contraintes. (toutes obligatoires, vérifiées en CI) :

1. multi-stage systématique ;
2. image finale ≤ 200 Mo par service ;
3. utilisateur non-root (USER 1001 ou similaire) ;
4. aucun secret en ENV ni en argument de build ;
5. tag d'image = SHA court du commit (git rev-parse --short HEAD) — le tag latest est interdit ;
6. exposition d'un endpoint /healthz joignable par les probes K8s ;
7. LABEL org.opencontainers.image.source pointant vers le dépôt Git.

Livrable. docker/Dockerfile.frontend et docker/Dockerfile.next (paramétré par --build-arg SERVICE=...).

Validation. trivy image --severity HIGH,CRITICAL --exit-code 1 <image> doit passer avec zéro vulnérabilité critique.

Pièges.

- next build requiert l'accès à internet ; ne le faites pas dans l'image runtime, uniquement dans le stage build ;
- réinstaller node_modules à chaque stage gonfle l'image et le temps de build — copiez-le avec COPY --from=build /app/node_modules ./node_modules ;
- npm ci est plus rapide et reproductible que npm install (nécessite un package-lock.json à jour).

Étape 6 — Premiers manifests : Namespace, Config, Secrets

Objectif. poser les fondations de configuration du namespace applicatif.

Contraintes.

- namespace = `salleenfrance` ;
- les valeurs non sensibles (URLs internes, niveau de log, mode d'environnement) vont dans une `ConfigMap` nommée `app-config` ;
- les valeurs sensibles (mot de passe Postgres, secret JWT, password Redis si vous en mettez un) vont dans un `Secret` nommé `app-secrets` ;
- aucun `Secret` ne doit être commité en clair dans Git — utilisez `SealedSecrets` ou, à défaut, un `Secret` généré dans la CI à partir de `GITHUB_SECRETS` ;
- chaque ressource porte les labels `app.kubernetes.io/name`, `app.kubernetes.io/part-of` : `salleenfrance`, `app.kubernetes.io/managed-by`.

Livrable. fichiers `k8s/base/00-namespace.yaml`, `k8s/base/01-configmap.yaml`, `k8s/base/02-secret.yaml` complétés (les # TODO du dépôt sont vos points d'entrée).

Validation.

- namespace visible dans le sélecteur en haut à gauche ;
- vues *Config / ConfigMaps* et *Config / Secrets* listent vos ressources ;
- les valeurs du `Secret` apparaissent masquées par défaut (œil pour révéler).

Pièges. un `Secret` K8s est base64, pas chiffré ; ne pas confondre encodage et chiffrement.

Étape 7 — Stateful : PostgreSQL

Objectif. déployer une instance PostgreSQL persistante, joignable par les services backend, et survivant à un `delete pod`.

Contraintes.

- `StatefulSet` (pas `Deployment`) ;
- 1 réplique ;
- `volumeClaimTemplates` produisant un PVC de 2 Gi sur la `storageClassName` : `standard` ;
- `Service headless` (`clusterIP` : `None`) nommé `postgres` ;
- script `seed/init.sql` monté dans `/docker-entrypoint-initdb.d/` via une `ConfigMap` ;
- `readinessProbe` et `livenessProbe` réelles (utilisez `pg_isready -U $POSTGRES_USER`) — pas de `tcpSocket` paresseux ;
- `resources.requests` et `limits` argumentés dans le rapport.

Livrable. `k8s/base/10-postgres.yaml`.

Validation.

- workload de type *StatefulSet* nommé postgres ;
- PVC associé en état Bound (vue *Storage / PVC*) ;
- PV correspondant créé dynamiquement ;
- dans le shell d'un pod backend (vue *Pod* → *Pod Shell*), `nc -zv postgres 5432` doit répondre `succeeded`.

Pièges.

- oublier le `volumeClaimTemplates` et tomber sur un `emptyDir` (data perdues à chaque restart) ;
- mettre une `livenessProbe` trop agressive : Postgres met du temps à démarrer la première fois (init scripts) — `initialDelaySeconds: 30` minimum.

Étape 8 — Cache : Redis

Objectif. déployer un Redis utilisé par `bookings-service` pour mettre en cache les disponibilités d'une salle (clé : `room:{id}:availability:{date}`).

Contraintes.

- Deployment (la perte du cache est acceptable, c'est un cache) ;
- 1 réplique ;
- Service ClusterIP nommé `redis`, port 6379 ;
- politique de mémoire : `maxmemory 128mb + allkeys-lru`, à passer via args ou via une ConfigMap montée en `redis.conf` ;
- pas de PVC.

Livrable. `k8s/base/11-redis.yaml`.

Validation.

- vue *Pod* → *Pod Shell* sur le pod `redis` : `redis-cli PING` répond `PONG` ;
- dans la vue *Pod* → *Logs*, vous voyez les commandes `GET room:...` arriver quand vous parcourez l'application.

Pièges. exposer Redis en NodePort (jamais).

Étape 9 — Microservices

Objectif. déployer les 4 services Next.js (`auth`, `sites`, `rooms`, `bookings`) et le frontend.

Contraintes. par service :

- Deployment, 2 répliques minimum (3 pour le frontend) ;
- `resources.requests: 100m CPU / 128 Mi RAM`. `resources.limits: 500m / 256 Mi` (à ajuster et justifier dans le rapport) ;
- variables d'env tirées du Secret et de la ConfigMap via `envFrom` ;
- `readinessProbe` HTTP sur `/api/healthz`, intervalle 5s ;
- `livenessProbe` HTTP sur `/api/healthz`, `initialDelaySeconds: 20` ;
- Service ClusterIP en face de chaque Deployment ;
- `PodDisruptionBudget` garantissant `minAvailable: 1` ;
- labels cohérents (`app.kubernetes.io/component: api` ou `frontend`).

Livrable. `k8s/base/2X-<service>.yaml` pour chaque service.

Validation.

- vue *Workloads / Deployments* : 4 deployments à 2/2 et 1 à 3/3 ;
- vue *Network / Services* : 5 services ClusterIP ;
- dans chaque pod, l'onglet *Environment* (à droite) montre les variables injectées par `envFrom` ;
- les *Events* d'un Pod ne contiennent aucune entrée `Unhealthy`.

Pièges.

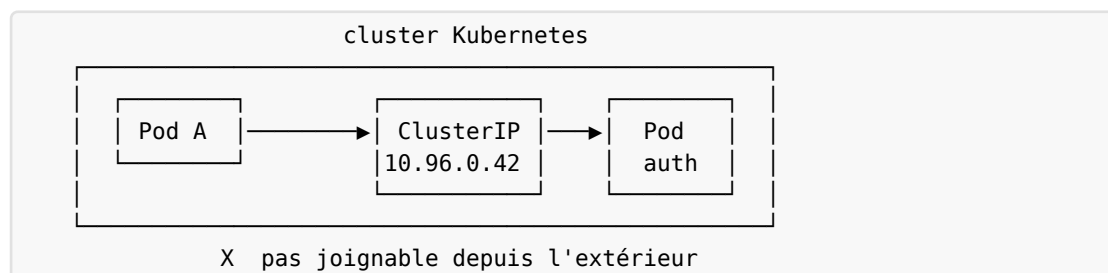
- même ressources partout par flemme — pénalisant ;
- `livenessProbe` qui tape une route coûteuse (DB) — boucle de redémarrage en cas de surcharge.

Intermède — Comprendre les types de Service

Avant d'attaquer l'Ingress, prenez 10 minutes pour bien fixer les quatre types de Service Kubernetes. C'est le piège classique en entretien — beaucoup de candidats les confondent.

Un Service Kubernetes est une abstraction réseau stable devant un ensemble de pods. Son rôle : donner aux pods une adresse fixe (IP virtuelle ou nom DNS) qui ne change pas même si les pods sous-jacents disparaissent / se recréent. Le type détermine comment cette adresse est exposée.

ClusterIP (par défaut)

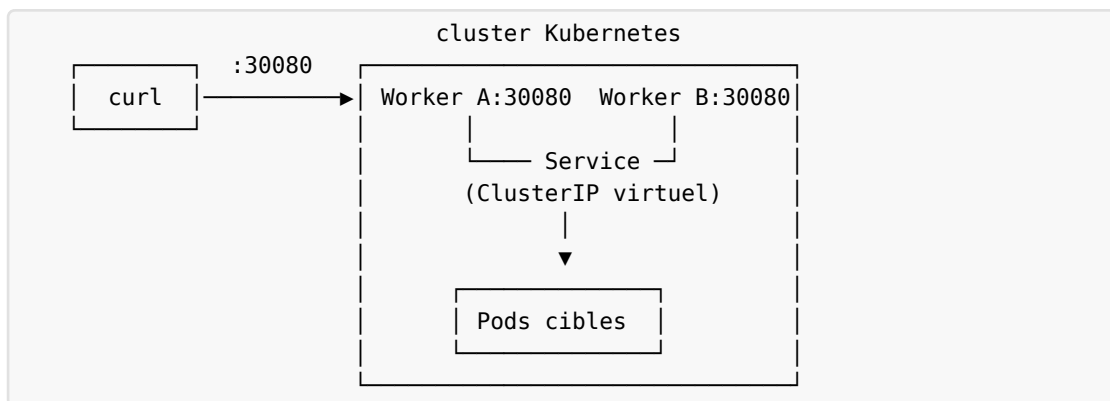


- IP virtuelle interne au cluster, inaccessible depuis l'extérieur.

- C'est la valeur par défaut d'un Service qu'on ne typifie pas.
- Usage : 99 % des services internes (microservices, DB, cache).

```
apiVersion: v1
kind: Service
metadata:
  name: auth-service
spec:
  # type: ClusterIP ← optionnel, c'est le défaut
  selector:
    app.kubernetes.io/name: auth-service
  ports:
    - { port: 3001, targetPort: 3001 }
```

NodePort



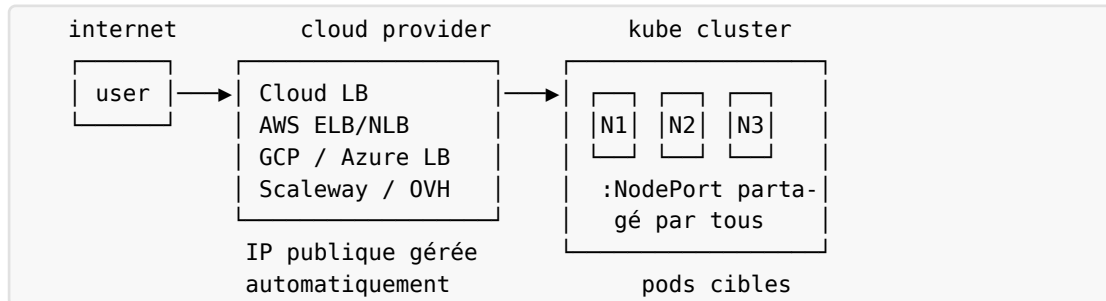
- Ouvrir un port (30000-32767 par défaut) sur chaque nœud du cluster.
- <n'importe-quelle-IP-de-nœud>:<NodePort> → forwardé vers le ClusterIP du Service → pods.
- Vous gérez vous-même la haute disponibilité en amont : si un nœud tombe, l'utilisateur qui pointait sur cette IP perd le service. Il faut un load balancer externe (DNS round-robin, HAProxy, F5...) pour distribuer.

```
apiVersion: v1
kind: Service
metadata:
  name: auth-service-nodeport
spec:
  type: NodePort
  selector:
    app.kubernetes.io/name: auth-service
  ports:
    - { port: 3001, targetPort: 3001, nodePort: 30080 }
```

Quand l'utiliser, vraiment ? Très rarement. En dev local pour exposer rapidement un service. En production, pour la couche bas-niveau d'un load balancer maison (HAProxy on-prem qui pointe vers les NodePorts).

Quand ne *jamais* l'utiliser. : pour exposer une base de données, un cache, un service interne. C'est l'erreur n° 1 des débutants.

LoadBalancer



- Demande automatiquement au cloud provider (AWS, GCP, Azure, Scaleway, OVH...) un load balancer externe avec une IP publique.
- C'est kube-controller-manager (en l'occurrence le cloud-controller-manager) qui pilote l'API du cloud pour provisionner le LB.
- Sous le capot, le LoadBalancer utilise des NodePorts sur chaque nœud — c'est l'extension naturelle du NodePort, mais industrialisée.
- Coûte de l'argent (un LB par Service, ~15-25 €/mois chez AWS/GCP).
- Ne fonctionne pas sur un cluster on-prem ou local (kind, minikube) sans un controller dédié (MetalLB, Cilium L2 Announce...).

```
apiVersion: v1
kind: Service
metadata:
  name: auth-service-lb
spec:
  type: LoadBalancer
  selector:
    app.kubernetes.io/name: auth-service
  ports:
    - { port: 443, targetPort: 3001 }
```

Une fois appliqué sur un cluster cloud, attendez quelques secondes : `kubectl get svc` affichera dans EXTERNAL-IP l'IP publique provisionnée par le cloud. C'est cette IP que vous mettez dans votre DNS.

ExternalName (mention)

- Pas un type d'exposition, mais un alias DNS interne vers un nom externe.
- auth-service interne → CNAME vers auth.partner.example.com.
- Aucun proxying : le pod reçoit juste le bon nom à résoudre.
- Usage : intégrer un service tiers comme s'il était dans le cluster.

Tableau de synthèse

Type	Visibilité	Qui assigne l'IP ?	Coût direct	Cas typique
ClusterIP	interne	K8s, IP du sous-réseau Service	0	tout service interne (DB, microservice...)
NodePort	depuis l'IP de chaque nœud, port 30000-32767	K8s, port choisi parmi le range	0	fallback, dev local, on-prem avec LB externe maison
LoadBalancer	publique	cloud provider	\$\$ (1 LB / svc)	exposition publique simple sur cloud
ExternalName	interne (DNS uniquement)	aucune (CNAME)	0	pointer vers un service externe

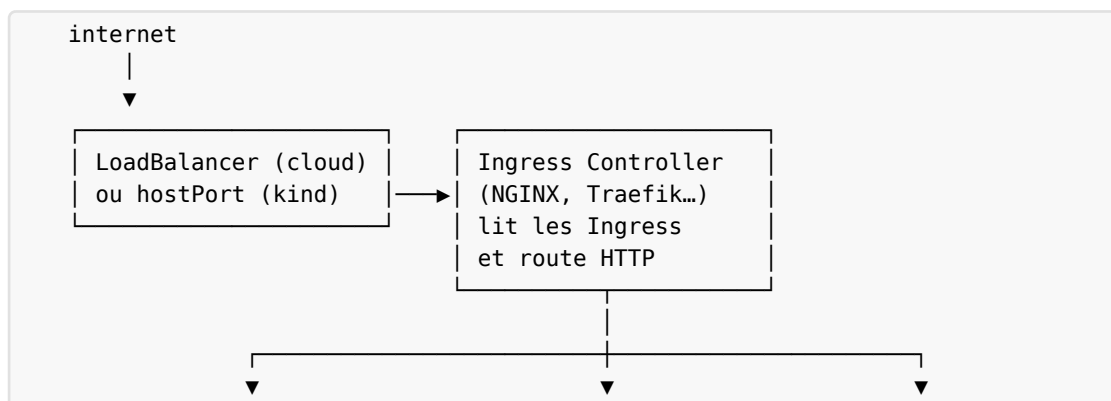
NodePort vs LoadBalancer — la confusion classique

Question	NodePort	LoadBalancer
Qui crée l'IP publique ?	personne — vous utilisez les IP des nœuds	le cloud provider, automatiquement
Sur cluster local (kind / minikube) ?	ça marche	❌ reste en Pending sans MetalLB
Combien d'entrées DNS à gérer ?	une par nœud (ou un LB en amont)	une seule, gérée par le cloud
Bascule automatique si un nœud tombe ?	❌ vous devez gérer (HAProxy, DNS round-robin)	géré par le cloud
Coût ?	gratuit	facturé par le cloud (~ 15-25 €/mois/svc)
Ports utilisables ?	30000-32767 (range par défaut)	n'importe lequel (le LB fait du NAT)

À retenir : le LoadBalancer est un NodePort + un load balancer cloud par-dessus. C'est une commodité offerte par le cloud, pas une primitive radicalement différente.

Et l'Ingress dans tout ça ?

Surprise : un Ingress n'est pas un type de Service. C'est une ressource séparée (étape suivante) qui s'appuie sur un Ingress controller (un Deployment qui tourne dans le cluster), lui-même exposé par un Service de type LoadBalancer (en cloud) ou NodePort (en local) ou hostPort (kind).





L’Ingress est une règle HTTP (host + path → backend). L’Ingress controller est l’implémentation (NGINX, Traefik, HAProxy, Istio Gateway...). On évite ainsi un LoadBalancer cloud par service — un seul LoadBalancer pour le controller, et le controller redirige par hostname / path.

Économie type : 30 services à exposer publiquement = 30 LoadBalancers à 25 € = 750 €/mois. Avec un Ingress + 1 LoadBalancer pour le controller : 25 € + le routage HTTP. Vous comprenez pourquoi tout le monde fait ça.

Étape 10 — Ingress

Objectif. exposer la plateforme sur un seul nom d’hôte avec routage par chemin, terminer le TLS au niveau de l’ingress.

Contraintes.

- IngressClass: nginx ;
- host = salleenfrance.local (ajout dans /etc/hosts) ;
- règles de routage :

Path	Backend
/api/auth(/\ \$(.*)	auth-service:3001
/api/sites(/\ \$(.*)	sites-service:3002
/api/rooms(/\ \$(.*)	rooms-service:3003
/api/bookings(/\ \$(.*)	bookings-service:3004
/	frontend:80

- certificat TLS produit par cert-manager (étape suivante) — vous pouvez d’abord tester en HTTP pur avant d’activer le TLS.

Livrable. k8s/base/40-ingress.yaml.

Validation.

- vue *Network / Ingresses* : 1 ingress, *Hosts* = salleenfrance.local, backends correctement résolus (icône verte) ;
- dans le navigateur ou avec curl http://salleenfrance.local/api/sites : réponse 200 (en HTTP — le TLS arrive à l’étape 11).

Pièges.

- deux approches valides, ne pas mélanger :
 - *pass-through* simple avec `pathType: Prefix` (les services reçoivent le path complet / `api/sites/...`) — recommandé pour ce TP ;
 - *regex + rewrite* avec `pathType: ImplementationSpecific` — exige les annotations `nginx.ingress.kubernetes.io/use-regex: "true"` et `nginx.ingress.kubernetes.io/rewrite-target;`
- oublier `/etc/hosts` → `curl: Could not resolve host: salleenfrance.local`.

Étape 11 — mTLS inter-services avec cert-manager

Objectif. sécuriser les flux internes entre microservices avec une authentification mutuelle TLS. Aucun service ne doit accepter de trafic d'un appelant qui ne présenterait pas un certificat client signé par la CA interne du cluster.

C'est l'objectif structurant du TP. La note finale dépend largement de la qualité de cette étape.

11.1 Mise en place de la PKI

Contraintes.

- une autorité de certification interne au cluster (Issuer `selfsigned-bootstrap` → Certificate `salleenfrance-ca` → `ClusterIssuer` `salleenfrance-ca-issuer`) ;
- tous les certificats applicatifs sont émis par `salleenfrance-ca-issuer` ;
- durée de vie certificat : 24h avec renouvellement automatique à 12h restantes (`renewBefore: 12h`) ;
- rotation testée et documentée.

Livrable. `k8s/base/50-pki.yaml`.

Validation.

- vue *Custom Resources* / *cert-manager.io* / *ClusterIssuer* : `salleenfrance-ca-issuer` Ready ;
- vue *Custom Resources* / *cert-manager.io* / *Certificate* : 1 ressource `salleenfrance-ca` Ready dans le namespace `cert-manager`.

11.2 Certificats par service

Contraintes.

- un Certificate par service ; le secret correspondant (`<service>-tls`) est monté en `read-only` dans le pod ;
- SAN minimum : `<service>.salleenfrance.svc.cluster.local, <service>.salleenfrance;`

- usages: [server auth, client auth, digital signature, key encipherment];
- Common Name = nom du service.

Livable. k8s/base/51-certs.yaml.

Validation. vue *Custom Resources / Certificate* : 5 certificats, tous Ready, tous avec un secret correspondant.

11.3 Activation côté serveur Next.js

Contraintes.

- chaque service Next.js démarre en HTTPS sur son port (3001 / 3002 / 3003 / 3004) en chargeant son certificat depuis /tls/tls.crt, /tls/tls.key ;
- requestCert: true, rejectUnauthorized: true côté serveur, avec la CA chargée depuis /tls/ca.crt ;
- les routes /api/healthz restent accessibles sans authentification client (sinon les probes K8s échoueraient) — utilisez un Service distinct <svc>-health en HTTP simple sur un port interne 8080 réservé aux probes ;
- la livenessProbe et la readinessProbe doivent cibler ce port 8080.

Livable. modification des Deployment et Service de la partie 9.

11.4 Activation côté client (appels inter-services)

Contraintes.

- les fetchs internes d'un service vers un autre incluent le certificat client + la CA :
 - chaque service a, monté, le certificat de sa propre identité (CN = son nom) ;
 - le trafic inter-services est toujours mTLS.

Livable. adaptation du code client (fourni en partie complète, à vous de connecter les chemins de fichiers via env vars).

11.5 Démonstration

Validation finale.

1. depuis un pod *non* équipé du certificat client (lancez un pod debug avec nicolaka/netshoot) : curl https://sites-service.salleenfrance:3002/api/sites doit échouer au handshake TLS (le message exact dépend du client : tls: bad certificate, alert bad certificate, alert handshake failure, SSL_ERROR_BAD_CERT_ALERT...);
2. depuis un pod équipé du certificat client : la même requête répond 200 ;
3. dans Freelens, vue *Custom Resources / Certificate*, vérifier la date d'expiration et déclencher un renouvellement manuel via cmctl renew.

Pièges.

- le pod cert-manager doit être Ready avant de créer les Certificate (mettre une dépendance Terraform ou un wait dans le pipeline) ;
- oublier d'inclure le certificat de la CA (ca.crt) côté client → unable to verify the first certificate ;
- se servir des probes K8s sur le port mTLS (elles n'ont pas de certificat client) — d'où le port 8080 dédié.

Étape 12 — CI/CD GitHub Actions

Objectif. industrialiser la livraison. Tout commit sur main doit produire un déploiement reproductible et sécurisé sur le cluster.

Contraintes.

- CI = GitHub Actions, 1 fichier par workflow dans .github/workflows/ ;
- workflow ci.yml (déclenché sur PR) :
 1. lint : yamllint, kubeconform, hadolint ;
 2. test : tests unitaires applicatifs (placeholders fournis) ;
 3. build : 5 images, taggées \${ github.sha } ;
 4. scan : trivy image — fail si CRITICAL ;
 5. sign : cosign sign --keyless ;
 6. push : ghcr.io/<org>/<service>:<sha> ;
- workflow cd.yml (déclenché sur push main) :
 1. récupère le kubeconfig du cluster cible (chiffré dans secrets.KUBECONFIG_B64) ;
 2. kubectl apply -k k8s/overlays/dev (Kustomize) ;
 3. kubectl rollout status sur chaque deployment ;
 4. test smoke curl sur /api/sites post-déploiement.

Livrable. .github/workflows/ci.yml, .github/workflows/cd.yml.

Validation.

- une PR ouverte avec un YAML invalide est bloquée au merge ;
- un push sur main produit un déploiement visible dans Freelens (rolling update — aucune interruption observée par un curl en boucle).

Pièges.

- mettre le KUBECONFIG en vars au lieu de secrets (fuite immédiate dans les logs) ;

- utiliser le tag `latest` dans les manifests — le rollout ne déclenche pas de mise à jour si le hash de l'image ne change pas ;
- `actions/checkout@v3` vs `@v4` — figez les versions.

Étape 13 — Observabilité, résilience, rolling updates

Objectif. prouver que la plateforme survit aux pannes et qu'on sait la diagnostiquer.

13.1 Casser pour comprendre

Manipulations à exécuter et à documenter dans le rapport (avec timestamps et observations Freelens) :

1. `kubectl delete pod -l app.kubernetes.io/name=bookings-service -n salleenfrance` — combien de temps avant retour à 2/2 ? Quel impact frontend ?
2. `kubectl drain <worker> --ignore-daemonsets` — où vont les pods ? Que se passe-t-il pour Postgres ?
3. Suppression du PVC Postgres (oui, vraiment) — état du StatefulSet ? Comment le récupérer ?

13.2 Mise à jour sans interruption

Modifier un message de réponse dans `sites-service`, rebuild l'image, recharger dans `kind`, déclencher `kubectl rollout restart deployment/sites-service`. En parallèle dans un autre terminal, lancer une boucle :

```
while true; do
  curl -sk -o /dev/null \
    -w "%{http_code}\n" \
    https://salleenfrance.local/api/sites
  sleep 0.2
done
```

Validation. aucun code 5xx ne doit apparaître pendant le rollout.

13.3 Logs

Mettre en place une commande qui suit en *tail* simultanément les logs des 4 services backend (`stern -n salleenfrance -l app.kubernetes.io/component=api`, ou `kubectl logs -f -l app.kubernetes.io/component=api -n salleenfrance --max-log-requests=10`).

Livrable. section *Observabilité* du rapport avec captures Freelens (vue *Pod* → *Logs*) et copies de la sortie `stern`.

Bonus

Choisissez au plus 2 bonus — la qualité prime sur la quantité.

- B1 — `HorizontalPodAutoscaler` sur `bookings-service` (CPU + custom metric `redis_cache_hit_ratio`) avec démonstration de charge.
- B2 — `NetworkPolicy` complète : Postgres n'accepte de connexions que des 4 services backend, le frontend ne joint que l'ingress.

- B3 — Surcouches Kustomize overlays/staging et overlays/prod avec différences réelles (réplicas, resources, hosts).
- B4 — Packaging Helm d'auth-service (chart + values.yaml) avec un test helm test.
- B5 — Service mesh allégé : remplacer le mTLS manuel par Linkerd ou Istio ambient ; comparer dans le rapport.
- B6 — Backup Postgres via CronJob + restauration prouvée.
- B7 — Mise en place de Velero pour snapshot/restore complet du namespace.
- B8 — Migration de kind vers un cluster cloud réel (Scaleway Kapsule, DigitalOcean) — le même Terraform doit fonctionner.

Annexe A — Cheatsheet kubectl

Contexte et namespace

```
kubectl config get-contexts          # lister les contextes
kubectl config use-context <ctx>     # basculer
kubectl config set-context --current --namespace=salleenfrance
kubectl get ns                       # lister les namespaces
```

Inspection

```
kubectl get all -n <ns>              # vue d'ensemble
kubectl get pods -o wide              # avec node + IP
kubectl get events --sort-by=.lastTimestamp # chronologie
kubectl describe pod <pod>           # détail (events à la fin !)
kubectl top pod                      # CPU/RAM (metrics-server requis)
kubectl top node
```

Logs

```
kubectl logs <pod>                  # logs du conteneur principal
kubectl logs <pod> -c <container>   # multi-conteneurs
kubectl logs <pod> --previous        # avant le dernier crash
kubectl logs -f -l app=auth-service # follow par label
stern -n salleenfrance .            # tail de tout le NS
```

Exécution / debug

```
kubectl exec -it <pod> -- /bin/sh
kubectl run debug --rm -it --image=nicolaka/netshoot -- bash
kubectl port-forward svc/postgres 5432:5432
kubectl cp <pod>:/etc/config.yaml ./config.yaml
```

Application et rollback

```
kubectl apply -f manifest.yaml
kubectl apply -k k8s/overlays/dev      # Kustomize
kubectl diff -f manifest.yaml          # avant d'appliquer
kubectl rollout status deployment/auth-service
kubectl rollout history deployment/auth-service
kubectl rollout undo deployment/auth-service
kubectl rollout restart deployment/auth-service
```

Scale et autoscale

```
kubectl scale deployment/auth-service --replicas=4
kubectl autoscale deployment/auth-service --min=2 --max=10 --cpu-percent=70
```

Stockage

```
kubectl get pv,pvc
kubectl describe pvc <pvc> # diagnostiquer un PVC Pending
```

Secrets et ConfigMaps

```
kubectl create secret generic app-secrets \
  --from-literal=JWT_SECRET=$(openssl rand -hex 32) \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl get secret app-secrets -o jsonpath='{.data.JWT_SECRET}' | base64 -d
kubectl create configmap app-config --from-file=./conf/ --dry-run=client -o
yaml
```

Diagnostic accéléré

```
kubectl get pods --field-selector=status.phase!=Running
kubectl get pods -o jsonpath='{range .items[*]}\n{.metadata.name}{":\t"}{.status.phase}{":\n"}{end}'
kubectl get pod <pod> -o yaml | grep -A5 lastState
```

cert-manager

```
cmctl status certificate <name>
cmctl renew <certificate>
kubectl get certificates,certificaterequests,orders,challenges -A
```

Ingress

```
kubectl get ingress -A
kubectl describe ingress <name> # voir les backends résolus
```

Nettoyage rapide

```
kubectl delete -k k8s/overlays/dev
kubectl delete pod --field-selector=status.phase=Failed -A
kubectl delete pod -l app=auth-service
```

Annexe B — Critères de réussite

Pour considérer le TP complet, vérifiez les points suivants :

- Outillage et dépôt — versions documentées, dépôt structuré, README à jour, .gitignore propre.
- Cartographie compose — schéma lisible, mapping compose↔K8s rigoureux, justifications.
- Cluster Terraform — apply reproductible, ingress + cert-manager installés, outputs exploitables.
- Containerisation — multi-stage, non-root, taille, scan trivy clean, tag par SHA.
- Manifests de base — namespace, ConfigMap, Secret bien typés, labels cohérents.
- Postgres stateful — StatefulSet, PVC bound, probes pertinentes, restart sans perte.

- Redis — déploiement clean, intégration applicative démontrée.
- Microservices — déployés, scalables, ressources justifiées, PDB.
- Ingress — routes correctes, TLS terminé, hosts résolus.
- mTLS — PKI cert-manager, certs par service, handshake refusé sans cert client, rotation prouvée.
- CI/CD GitHub Actions — workflow lint + build + scan + sign + push + deploy, fail propre, rollout sans coupure.
- Observabilité / résilience — manipulations documentées, logs centralisés, rolling sans 5xx.
- Rapport RAPPORT.md — clarté, justifications, schéma, captures Freelens.

Annexe C — Pièges classiques (à lire avant de commencer)

1. Tag latest. Interdit. Tags par SHA. Sinon le rollout ne se déclenche pas.
2. Pas de ressources.requests. K8s ne peut pas planifier intelligemment.
3. livenessProbe == readinessProbe. La readiness retire du Service ; la liveness redémarre. Confondre les deux fait redémarrer un pod qui n'est que lent.
4. Secret commités en clair. Utiliser SealedSecrets ou un coffre.
5. Postgres en Deployment. Pas de garantie d'identité stable, comportement imprévisible avec un PVC.
6. emptyDir au lieu d'un PVC. Données perdues à chaque restart du pod.
7. mTLS oublié sur les probes. La probe ne présente pas de cert ; le pod paraît Unhealthy même quand il fonctionne. Réservez un port /healthz non-mTLS.
8. NodePort pour exposer Postgres ou Redis. Jamais. Gardez les datastores en ClusterIP.
9. namespace: default. Tout doit aller dans salleenfrance.
10. CI qui déploie sans kubectl rollout status. Le job se termine avant que le rollout ne soit fini → smoke tests cassés sans raison apparente.
11. kind load docker-image oublié. ErrImagePull immédiat car le cluster ne voit pas le registre local.
12. Freelens ouvert sur le mauvais cluster. Toujours vérifier le sélecteur en haut à gauche avant de croire à un bug.